

Manual Test Checklist

ECITY — Mobile Shop Management Web App

Version 1.1 · 2026

What a person still has to judge, module by module

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

How to Use This

Automated tests cover the mechanics — 266 unit and integration tests, 564 browser tests across four screen sizes. This checklist covers what a person still has to judge: does it *feel* right, does the wording make sense, does the shop's actual workflow survive contact with the screen.

Work through a module's section after it is built, and again before go-live. Tick as you go. Anything that fails is a bug, not a note.

One section per module. M1–M14 append to this file as they ship.

Before you start

```
docker compose up -d      # or open Postgres.app
npm run db:migrate
npm run db:seed           # resets the three test accounts
npm run dev               # http://localhost:3000
```

To return to a known-clean state at any point, re-run `npm run db:seed`. It upserts, so it is safe to repeat.

Test accounts

Email	Password	Role	Branch scope
admin@ecity.local	ChangeMe!2026	Admin / Owner	All branches (14 permissions)
manager@ecity.local	ChangeMe!2026	Branch Manager	MAIN only (7 permissions)
staff@ecity.local	ChangeMe!2026	Staff	MAIN only (3 permissions)

ChangeMe!2026 is only the fallback the seed uses when `SEED_PASSWORD` is not set — which is the normal case on your own machine. **On staging or any deployed server the password is whatever `SEED_PASSWORD` was set to** in that server's `.env`; the accounts above are otherwise identical. Read it back with `grep SEED_PASSWORD ~/app/.env`.

Use a **private/incognito window** to hold a second session, so you can be two users at once without signing out.

These accounts must never exist in production. They are flagged `mustChangePassword` and are created only by the seed.

M0 — Foundations

Delivers. PRD FR-1.1 – FR-1.5, FR-31.1, plus branch context and the authorisation layer everything else depends on.

Signing in as the admin exercises perhaps a tenth of this module. The interesting part is what the *other two* users cannot do.

1. Authentication

Start signed out, at `/login`.

- [] Wrong password for a real account → *"Email or password is incorrect."*
- [] A made-up email with any password → **the identical message**. It must not reveal whether the account exists
- [] Empty email or password → inline validation, no request sent

- [] Visit `/settings/users` directly while signed out → redirected to `/login`
- [] Correct credentials → lands on `/dashboard`
- [] Avatar menu → **Sign out** → back at `/login`
- [] After signing out, the browser Back button does **not** restore the app
- [] Sign in again in a second browser → both sessions work independently

Password reset

Email is not wired up until M14, so the reset link is shown on screen in development.

- [] `/forgot-password` → enter `admin@ecity.local` → confirmation appears
- [] A **yellow development box** shows the reset link
- [] Enter an address that does not exist → **the same confirmation**, no hint that the account is unknown
- [] Follow the link, set a new password → redirected to `/login`
- [] The **old** password no longer works
- [] The **new** password works
- [] Open the same reset link a second time → refused as already used
- [] Reset the password back to `ChangeMe!2026`, or re-run `npm run db:seed`

Logout

- [] Get the password wrong 8 times in a row → *"Too many failed attempts."*
- [] Wait 15 minutes, or `npm run db:seed`, to clear it

2. Branch scoping

The heart of M0. Sign in as each user and open the branch switcher in the header.

- [] **admin** — sees *All branches (consolidated)*, MAIN and NORTH
- [] **manager** — sees **MAIN only**. No NORTH, and no "All branches"
- [] **staff** — sees MAIN only
- [] As admin, switch between branches → the dashboard's *Branch context* card updates (active branch, accessible count, consolidated yes/no)
- [] The chosen branch survives a page reload
- [] On a phone-width window the switcher shows the branch **code**, not the full name, and the header still fits

3. Permissions

Check what is **missing**, not what is present.

User	Sidebar should show
admin	Dashboard, Users, Roles, Audit log
manager	Dashboard, Users, Audit log — no Roles
staff	Dashboard only

- [] Each user's sidebar matches the table
- [] As **staff**, type `/settings/roles` into the address bar → redirected, not shown
- [] As **staff**, type `/settings/users` → redirected
- [] As **manager**, `/settings/roles` → redirected
- [] The dashboard's *Your permissions* card lists 14 / 7 / 3 entries respectively

4. Server-side authorisation

A hidden menu item is not a control. This step proves the server refuses the request, not just that the button was hidden.

Sign in as **staff** in the browser, then copy the session cookie from DevTools → Application → Cookies → `ecity_session`, and run:

```
COOKIE='ecity_session=PASTE_VALUE_HERE'
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/api/users -H "Cookie: $COOKIE"
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/api/audit -H "Cookie: $COOKIE"
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/api/roles -H "Cookie: $COOKIE"
```

- [] All three return **403**, not data
 - [] The body reads `{"error":"...", "code":"FORBIDDEN"}`
 - [] With no cookie at all → **401**
 - [] As **manager**, forcing another branch is refused:
- ```
curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:3000/api/session/branch \
-H "Cookie: $COOKIE" -H 'content-type: application/json' -d '{"branchId":2}'
```
- [ ] Returns **403** — branch NORTH is not theirs
  - [ ] `{"branchId":null}` (the consolidated view) also returns **403**

## 5. User management

As **admin**, Users → Add user.

- [ ] Password under 10 characters → rejected inline
- [ ] An email that already exists → *"A user with this email already exists."*
- [ ] No role chosen → *"Choose a role."*
- [ ] Create a Staff user assigned to **NORTH** only
- [ ] They appear in the user list with the right role and branch badge
- [ ] Sign in as the new user → branch switcher shows **NORTH only**
- [ ] Their first sign-in is flagged "must change password"

### Roles

As **admin**, Settings → Roles.

- [ ] Select **Staff**, tick *View the audit log*, Save
- [ ] Sign in as staff → **Audit log now appears** in their sidebar
- [ ] Any staff session open elsewhere was **signed out** when the role changed, rather than keeping stale permissions until next login

- ☐ Untick it and Save again
- ☐ Try to remove *Create and edit users* from **Admin** → refused, so the business cannot lock itself out
- ☐ System roles cannot be deleted

## 6. Audit log

As **admin**, Settings → Audit log.

- ☐ Everything done above appears: `LOGIN`, `LOGIN_FAILED`, `LOGOUT`, `CREATE`, `PERMISSION_CHANGED`, `PASSWORD_RESET_COMPLETED`
- ☐ Each row shows who, what, when
- ☐ Expanding a "*n field(s)*" row shows old → new values
- ☐ A password hash **never** appears in a changes payload
- ☐ Paging works if there are more than 50 entries

The log is append-only, enforced by the database rather than by code:

```
psql ecity -c "update audit_log set summary='tampered' where id=1;"
psql ecity -c "delete from audit_log where id=1;"
```

- ☐ Both fail with `audit_log is append-only`

## 7. Responsive layout

Resize the window, or use DevTools device mode. Check at ~375px, ~810px and full width.

- ☐ Below ~768px the sidebar is replaced by a **hamburger** top-left
- ☐ Tapping it opens a drawer; choosing a destination closes it
- ☐ On **Users**, the table becomes **one card per user**
- ☐ On **Audit log**, entries become cards
- ☐ Nothing scrolls sideways at any width
- ☐ Buttons and links are comfortably tappable with a thumb
- ☐ The **Roles** editor is usable — role list becomes a horizontal strip
- ☐ Forms stack, with full-width buttons

## 8. Theme

- ☐ Primary colour is **black** throughout (buttons, active nav item)
- ☐ Set `APP_THEME=navy` in `.env`, restart → the whole app is blue, with no other visual change
- ☐ An invalid value such as `APP_THEME=nonsense` → falls back to black rather than breaking
- ☐ Set it back to `onyx`

## What M0 deliberately does not include

Do not raise these as bugs:

- No products, inventory, IMEIs, customers or suppliers — those are M1 and M2
- The dashboard shows branch context and permissions, **not** sales figures. The real dashboard is M10
- No branch create/edit screen — M1. The two seeded branches are fixed
- Password reset emails print to screen instead of sending — M14
- Notifications, reports, exports — later modules

## M1 — Master Data

**Delivers.** PRD FR-2.1 – FR-2.4, FR-3.1, FR-3.2, FR-3.8, FR-5.10, FR-6.6, FR-10.1.

The shop gets configured and the people it trades with exist. Nothing here moves stock or money yet — these are the pickers and directories the rest of the system draws on.

**Customer** = someone who buys from the shop. **Supplier** = someone the shop buys from. Both are shared across every branch.

### 1. Business settings

Sign in as **admin**, go to **Settings** → **Business**.

- [ ] Four tabs: Profile, Tax, Payments, Expenses
- [ ] On a phone the tabs scroll sideways rather than wrapping

#### Profile

- [ ] Edit the business name, save, reload — the change persisted
- [ ] Enter a malformed GST number → *"Enter a valid 15-character GSTIN."*
- [ ] Enter a valid one (29ABCDE1234F1Z5) → accepted, stored upper-case
- [ ] Currency is fixed at INR and cannot be edited
- [ ] Toggle **Prices include tax** → the explanation below it changes
- [ ] Save, reload → the toggle held its new position

#### Tax

- [ ] Five GST slabs are listed: 0, 5, 12, 18, 28%
- [ ] **GST 18%** is marked Default
- [ ] Rates display as 18.00%
- [ ] Add a rate named **GST 3%** at 3 → appears in the list
- [ ] Add one with the same name → refused as a duplicate
- [ ] Deactivate a non-default rate → shows Inactive
- [ ] Try to deactivate the **default** rate → refused, with the reason

#### Payments

- [ ] Cash, UPI, Card, Bank Transfer are listed with their type badge
- [ ] Cash carries a **Cash drawer** badge; the others do not
- [ ] Deactivate UPI → shows Inactive; reactivate it

- [ ] Try to deactivate **Cash** → refused: at least one cash method must stay active

## Expenses

- [ ] Seven categories: Rent, Electricity, Salaries, Transport, Packaging, Repairs, Miscellaneous
- [ ] Add one → appears immediately
- [ ] Add the same name again → refused

## 2. Branches

**Settings** → **Branches** as admin.

- [ ] MAIN and NORTH are listed with manager and user counts
- [ ] **Add branch** — code **MGROAD**, name **MG Road** → created and listed
- [ ] The code is forced to upper case even if typed lower
- [ ] Add another with code **MGROAD** → *"Branch code MGROAD is already in use."*
- [ ] A code containing a space or ! is rejected inline
- [ ] Open the new branch, change its name, save → the list shows the new name
- [ ] Assign a manager from the dropdown → shows in the Manager column
- [ ] Leave the manager empty → **saves without error** (*this failed silently before — see §5*)

### Deactivation — the rule that matters (FR-3.8)

- [ ] Deactivate **MG Road** → badge turns Inactive, **the row stays**
- [ ] Open the header **branch switcher** → MG Road is **gone**
- [ ] Reactivate it → it comes back in the switcher
- [ ] Deactivate every branch but one, then try the last → refused: at least one branch must stay active

## 3. Customers and suppliers

**Customers** in the sidebar.

- [ ] The list is empty, or holds only what you have created
- [ ] **Add customer** with a name only → saves. A walk-in has nothing else
- [ ] Add one with phone **9876543210**
- [ ] Add a second with the **same phone** → *"Another customer (...) already uses this phone number."*
- [ ] Give a **supplier** that same phone → **allowed**, they are different directories
- [ ] Edit a customer, keep their phone unchanged, save → no false duplicate
- [ ] Enter **not-a-phone** → rejected inline
- [ ] Enter **MiXeD@Example.COM** → stored lower-case (check in Studio)

### Search and status

- [ ] Search by part of a name → matches
- [ ] Search by phone → matches
- [ ] Search by email → matches

- [ ] Search by GST number → matches
- [ ] **Deactivate** a customer → disappears from the default list
- [ ] Click **Show inactive** → they reappear, badged Inactive
- [ ] Reactivate → back to normal

Repeat the same pass on **Suppliers**. Suppliers additionally have a **Company** field; customers do not.

## 4. Permissions

| User    | Should see in the sidebar                                                       |
|---------|---------------------------------------------------------------------------------|
| admin   | Customers, Suppliers, Business, Branches, Users, Roles, Audit log               |
| manager | Customers, Suppliers, Business, Users, Audit log — <b>no Branches, no Roles</b> |
| staff   | Customers, Suppliers — <b>no Settings group at all</b>                          |

- [ ] Each user's sidebar matches
- [ ] As **staff**, open </settings/business> directly → redirected
- [ ] As **staff**, open </settings/branches> directly → redirected
- [ ] As **staff**, open </customers/new> directly → redirected (they may view customers, not create them)
- [ ] As **manager**, </settings/branches> → redirected. Managers run a branch; they do not create them

At the API, signed in as staff (see M0 §4 for how to get the cookie):

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/api/branches?manage=1 \
-H "Cookie: $COOKIE"
curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:3000/api/customers \
-H "Cookie: $COOKIE" -H 'content-type: application/json' -d '{"name":"Sneaky"}'
```

- [ ] The POST returns **403**

## 5. Form errors are never silent

A bug found during M1: a validation failure with no visible message meant the Save button appeared to do nothing.

- [ ] Paste 300 characters into any address field → a visible error appears
- [ ] On every form, an invalid field shows red text under **that** field
- [ ] No form ever fails with the button simply doing nothing

## 6. Audit

**Settings** → **Audit log** as admin.

- [ ] Everything above is recorded: branch created, customer created, tax rate added, payment method deactivated
- [ ] Entity types include [branch](#), [customer](#), [supplier](#), [tax\\_rate](#), [payment\\_method](#), [business](#)
- [ ] Expanding a row shows old → new values



## 7. Responsive

---

At ~375px, ~810px and full width:

- [ ] Customers, Suppliers and Branches lists become **cards** below 768px
- [ ] Business settings tabs scroll rather than wrap
- [ ] Forms stack to one column; buttons go full width
- [ ] Nothing scrolls sideways
- [ ] The sidebar's **Master data** and **Settings** groups appear in the mobile drawer

## What M1 deliberately does not include

---

- **No customer/supplier history tab yet.** Clicking a customer opens the edit form. The profile view showing their purchases, returns and credit arrives with M4/M5, when there is something to show. *This is a known gap against M1's stated criteria, carried into M2.*
  - No products, stock or IMEI's — that is M2
  - Attachments have a working API and storage layer, but **no upload button on any screen yet**; the first screen that needs one is M3's purchase form
  - Logo upload is not wired to the business profile
- 

## M4 — Sales & Billing

**Delivers.** PRD FR-6.1 – FR-6.8, FR-26.1, FR-26.3, FR-26.4, FR-38.2, FR-38.3.

This is the screen the shop lives on. Everything before it existed so that this one could work: a bill is built, money is taken, stock drops, and an invoice comes out. Test it standing up, at speed, the way the counter will.

**Before you start.** You need stock. Run through M3 once to bring in at least one USED or ER handset with an IMEI, and one accessory with a quantity of ten.

### 1. Speed at the counter

---

Go to **Billing**.

- [ ] The search box already has focus — you can start typing without clicking
- [ ] Type an accessory name → results appear as you type
- [ ] With exactly one result, pressing **Enter** puts it on the bill
- [ ] Scan or paste an IMEI → that exact handset lands on the bill
- [ ] **Time yourself:** accessory → Cash → Save, keyboard only, under 20 seconds
- [ ] The whole flow needs no mouse

### 2. The bill

---

- [ ] A device line shows the IMEI or serial underneath the product name
- [ ] A device line shows its main type (USED / ER / ACT / GLOBAL)

- [ ] A GLOBAL device also shows **NEW CUT** where that applies
- [ ] A device line's quantity is fixed at 1 and cannot be edited
- [ ] An accessory line's quantity can be raised, and the total follows
- [ ] Adding the same IMEI twice is refused with a clear message
- [ ] Line discount reduces that line only; bill discount reduces the total
- [ ] **Staff** signed in cannot give a discount at all
- [ ] Totals: taxable + GST equals the total, exactly, to the paisa
- [ ] Change a quantity and the totals update instantly

### 3. The bill survives real life

---

- [ ] Build a half-finished bill and **reload the page** → it is still there
- [ ] Clear bill empties it, and asks nothing you would regret confirming
- [ ] Switch branch with a bill open → the bill clears (it belonged to the other branch's stock)

### 4. Payment

---

- [ ] **+ Cash** fills the full balance in one press
- [ ] Split across two methods (say Cash 500 + UPI rest) → both are recorded
- [ ] Pay less than the total with **no customer attached** → refused, because an unpaid balance needs someone to owe it
- [ ] Attach a customer, then save partly paid → the sale shows **PARTIAL**
- [ ] Overpaying is refused

### 5. Customer, without leaving the bill

---

- [ ] Press **New** beside the Customer box
- [ ] Name and phone only → **Add to bill**
- [ ] The new customer is created *and* attached, and the bill is untouched
- [ ] A duplicate phone number is refused with a message naming the clash
- [ ] Cancel closes the dialog and changes nothing

### 5c. Pickers do not silently truncate

---

Customers, suppliers and products are chosen through a **searchable picker**, never a fixed dropdown — a capped list drops records with no hint that it did.

- [ ] The Customer box on billing opens a search, not a long dropdown
- [ ] Typing a phone number finds the customer
- [ ] A customer created minutes ago is findable
- [ ] Same for **Supplier** on the purchase form

- ☐ Same for **Product** on the purchase form and on Add device

## 5b. Customer history (FR-6.7)

---

Open **Customers** and click a customer who has bought something.

- ☐ The **History** tab opens first — that is what you came to see
- ☐ Total spend, number of purchases, outstanding and last purchase date
- ☐ Every invoice is listed, newest first, with branch and item count
- ☐ Clicking an invoice number opens that sale
- ☐ Purchases from **every branch** appear, not just the current one
- ☐ A part-paid bill shows **PARTIAL**, and outstanding is non-zero and red
- ☐ A voided bill is listed but does **not** count towards spend or dues
- ☐ A customer who has bought nothing says so, rather than showing an empty table
- ☐ The **Details** tab still edits the customer and saves
- ☐ On a phone the history becomes cards and nothing overflows sideways

## 6. The invoice

---

Save a bill; you land on the sale.

- ☐ The invoice shows shop name, GSTIN, branch, invoice number and date
- ☐ Customer details appear, or "Walk-in customer"
- ☐ Every line shows quantity, price, tax and amount
- ☐ GST is broken out per rate
- ☐ **Print A4** produces a clean full-page invoice with no navigation on it
- ☐ **Print receipt (80 mm)** produces a narrow slip that fits the thermal roll
- ☐ **Download PDF** saves a file named after the invoice number
- ☐ Open the PDF: every figure matches the screen, and no character shows as a blank box
- ☐ The PDF's amounts are labelled INR (the rupee glyph is deliberately not used — see <src/server/pdf/invoice-pdf.tsx>)
- ☐ **Share** hands the PDF to the phone's share sheet (WhatsApp, mail); on a desktop browser it copies the invoice link instead

## 6b. The invoice is a statutory GST document (OQ-4)

---

First set **Settings** → **Business** → **GST state**, and a **GST state** on each branch. Give a few products an **HSN code** (8517 phones, 8544 cables).

- ☐ The invoice is headed **Tax Invoice**
- ☐ **Place of supply** appears, with the state code and name
- ☐ Each line shows its **HSN** code

- ☐ An ordinary local sale shows **CGST** and **SGST** as separate lines, each half the rate — never a single combined "GST 18%"
- ☐ CGST + SGST adds up to exactly the tax shown
- ☐ An **HSN summary** table appears below the totals on A4
- ☐ The HSN summary's taxable and tax columns add up to the invoice totals
- ☐ Set a customer's **GST state** to another state, bill them → the invoice shows a single **IGST** line and no CGST/SGST
- ☐ The customer pays the same total either way — only the split changes
- ☐ A product with no HSN still bills; the line shows "—" and stays out of the summary
- ☐ The downloaded PDF matches the screen on every one of these

## 7. Stock actually moved

---

- ☐ The handset you sold is now **Sold** on the device page
- ☐ Its IMEI no longer appears in billing search
- ☐ The accessory's branch stock dropped by the quantity sold
- ☐ The stock ledger shows one **SALE** row referencing the invoice

## 8. The other billing system (FR-38.2, FR-38.3)

---

- ☐ A **NEW** handset never appears in billing search at all
- ☐ Forcing one onto a bill is refused server-side too
- ☐ Open that device → **Sold in other system** is offered
- ☐ Mark it → status becomes **SOLD\_PENDING\_IMPORT** and stock drops now
- ☐ It disappears from billing search, so it cannot be sold twice
- ☐ An ECITY-channel device does **not** offer that button

## 9. Sales list

---

- ☐ Every saved sale is listed, newest first
- ☐ Search by invoice number, customer name or IMEI finds it
- ☐ Payment status filter: PAID / PARTIAL / UNPAID each return the right rows
- ☐ Branch filter narrows to that branch; a branch-scoped user sees only theirs
- ☐ From/To dates include both end days
- ☐ Filters survive a page reload (they are in the URL, so they are shareable)

## 10. Concurrency — needs two browsers

---

- ☐ Sign in on two windows, put the **same IMEI** on both bills
- ☐ Save both → the first succeeds, the second fails with a clear message

- [ ] The device is Sold exactly once, and only one sale exists
- [ ] Save a bill, then press Save again on a restored copy → still one sale

## 11. Responsive

---

- [ ] On a phone the billing screen is usable one-handed; nothing overflows sideways
- [ ] The cart and the totals panel stack rather than squeeze
- [ ] The invoice is readable on a phone
- [ ] On a large screen the search, cart and payment panel sit side by side

## What M4 deliberately does not include

---

- **No returns or exchanges** — that is M6
  - **No trade-in** on the bill — also M6
  - **No Excel import** of the other system's sales — that is M12; devices marked `SOLD_PENDING_IMPORT` are waiting for it
  - **Returns do not appear in customer history yet** — the tab says so; they arrive with M6. Payments now have their own Statement and Receipts tabs
  - No daily cash-up or Z-report — M7
- 

## Cross-cutting — Lists & pagination

Every list in the app is paginated server-side at **25 rows a page**. Check these on **Products, Devices, Customers, Suppliers, Purchases, Sales** and the **Audit log** — the control is the same one everywhere.

- [ ] Under each list: *"Showing 1–25 of N"*, with the real total
  - [ ] A list with more than 25 rows offers **Previous / Next**
  - [ ] A list with 25 or fewer offers no page buttons, but still states the count
  - [ ] On page 1, **Previous** is greyed out and does nothing when clicked
  - [ ] On the last page, **Next** is greyed out
  - [ ] **Next** advances, and the rows are genuinely different
  - [ ] Page 1's URL has no `?page=` on it — one address for the first page
  - [ ] Apply a filter, then page forward → **the filter is still applied**
  - [ ] Change a filter while on page 3 → you go back to page 1, not an empty page
  - [ ] Copy a page-2 URL into a new tab → same page, same filters
  - [ ] Browser back returns to the previous page of results
  - [ ] On a phone the buttons are big enough to tap and do not overflow sideways
- 

## M5 — Payments, Credit Sales & Customer Dues

**Delivers.** PRD FR-7.1 – FR-7.6.

Credit is where a shop quietly loses money. This module exists so that the answer to "who owes us what, and since when?" is one screen and always right.

**Before you start.** Set **Settings** → **Business** → **Default credit period** (30 days is the default). You need one product in stock and one customer.

## 1. A bill can leave the counter unpaid

---

- [ ] Bill something to a **named customer** and pay nothing → the **Credit terms** panel appears, with a due date already filled in
- [ ] Pay the bill in full → the credit panel disappears (a cash sale needs no due date)
- [ ] Pay part of it → the panel comes back
- [ ] Try to leave a balance with **no customer** → refused, because a walk-in cannot be given credit
- [ ] Change the due date by hand and save → the sale shows that date
- [ ] Save unpaid → the sale shows **UNPAID**; part-paid shows **PARTIAL**
- [ ] Open that sale: it shows **Outstanding**, the **due date**, your credit note, and a **Collect payment** button
- [ ] Past the due date, the sale says *overdue* in red
- [ ] A fully paid sale shows none of that panel

## 2. Customer dues (FR-7.6)

---

Open **Sell** → **Customer dues**.

- [ ] Your unpaid customer is listed, with the amount owed
- [ ] **Total outstanding** matches the sum of the rows
- [ ] The **By age** panel adds up to the same total
- [ ] A bill created today sits in the **0–7 days** column
- [ ] Search by customer name and by phone number
- [ ] Filter by branch → only that branch's bills count
- [ ] **Overdue only** hides a bill whose due date has not passed yet
- [ ] A customer who has paid everything drops off the list entirely
- [ ] On a phone the rows become cards and nothing scrolls sideways

## 3. Collecting money (FR-7.3)

---

- [ ] Press **Collect** on a row → the customer's open bills are listed, oldest first
- [ ] Type an amount → it is allocated down the list, oldest first, and you can see exactly which bills it lands on before saving
- [ ] **Settle everything** fills in the exact total owed
- [ ] **Choose invoices** lets you allocate by hand instead
- [ ] Allocating more than a bill owes is refused, and says which bill
- [ ] Save → you land on a printable receipt with its own number

- ☐ The bill is now **PAID** on the sales list *and* on the sale itself
- ☐ Three part payments settle a bill and flip it to Paid on the third

## 4. Paying at a different branch (FR-7.5)

---

- ☐ Bill something on credit at **Branch A**
- ☐ Collect the money at **Branch B** (change *Collected at* on the form)
- ☐ The receipt shows Branch B; the invoice still shows Branch A
- ☐ The statement shows both, each against its own line

## 4b. Branch-wise (FR-7.6)

---

The **By branch** panel appears on the dues screen when there is more than one branch with activity.

- ☐ Each branch shows outstanding, overdue, collected and receipt count
- ☐ The **All branches** row adds up the columns above it
- ☐ Outstanding here matches the total at the top of the page
- ☐ Bill on credit at Branch A, collect at Branch B → A's *outstanding* falls, B's *collected* rises. They are not supposed to match
- ☐ Change the **Collected from / to** dates → only the collected column moves; what is owed is owed regardless of the window
- ☐ Void a receipt → it stops counting as collected, and the debt returns to the branch that raised the bill

## 5. The receipt (FR-26.2)

---

- ☐ Shop name, branch, receipt number, date, customer
- ☐ The amount, the method, and the reference if you gave one
- ☐ **Applied to** lists each invoice and how much went to it
- ☐ Money beyond what was owed shows as **held on account as an advance**
- ☐ **Balance after this receipt** matches the customer's account
- ☐ Print A4 and Print receipt (80 mm) both produce clean output

## 6. Voiding a receipt

---

- ☐ **Void** is offered to the owner and manager
- ☐ It is **not** offered to counter staff — they may collect, not erase
- ☐ Voiding needs a reason
- ☐ After voiding: the receipt is marked VOIDED but still on record
- ☐ The invoice it settled is **UNPAID** again
- ☐ The customer's balance has gone back up
- ☐ The statement shows a **Reversal** line — the old line is untouched

- ☐ Voiding the same receipt twice is refused

## 7. The statement (FR-7.6)

---

Open a customer → **Statement**.

- ☐ Every movement, oldest first, with a running balance
- ☐ An invoice appears as a debit; a payment as a credit
- ☐ Clicking an invoice number opens that sale; a receipt opens that receipt
- ☐ The closing balance matches the figure at the top of the customer page
- ☐ A customer in credit shows a negative balance labelled *in credit*

## 8. The numbers reconcile

---

This is the one that matters. Pick any customer with dues:

- ☐ Add up what their open invoices still owe, by hand
- ☐ Compare with the balance on the dues list — they must be identical
- ☐ Compare with the statement's closing balance — also identical
- ☐ Void a receipt and repeat: all three still agree

## What M5 deliberately does not include

---

- **No returns or exchanges** — M6. A refund is not a negative payment
- **No cash drawer or daily closing** — M7. Money collected is recorded, but the day is not yet reconciled
- **No statements by email or WhatsApp** — M13
- **No interest, credit limits or dunning letters** — not in v1 at all

# M6 — Returns, Exchange & Trade-In

**Delivers.** PRD FR-8.1 – FR-8.5, FR-9.1 – FR-9.3, plus correcting a device.

Goods come back and old phones come in. The rule underneath everything here: **a returned handset is never immediately sellable again.**

**Before you start.** You need a completed sale with a handset on it and another with an accessory. Sign in as **admin** unless a step says otherwise.

## 1. Taking a return (FR-8.1)

---

- ☐ **Sell** → **Returns** → **Take a return** offers one search box
- ☐ Find a bill by its **invoice number**
- ☐ Find the same bill by **customer name**
- ☐ Find it by the **IMEI** of a handset on it



- [ ] Opening a sale and pressing **Take a return** gets you to the same place
- [ ] Only lines with something left to return are offered
- [ ] Returning 2 of 5 gives back exactly two fifths of that line
- [ ] Asking for more than remains is refused, and says how many are left
- [ ] Return 3 of 4, then try to return 2 more → refused, 1 left
- [ ] A whole bill returned at once is marked **FULL**; part of one is **PARTIAL**
- [ ] Goods can be returned to a **different branch** than they were sold at

## 2. A returned handset is not sellable (FR-8.2)

---

This is the one most likely to be got wrong. Test it properly.

- [ ] Return a handset → it appears in **Returns** → **Inspection queue**
- [ ] Search that IMEI at the till → **nothing comes up**
- [ ] The device page shows status **RETURNED**
- [ ] An accessory behaves differently: it goes straight back into branch stock and can be sold again immediately

## 3. Inspection (FR-8.3)

---

- [ ] **Inspect** offers Available, Used, Damaged, Repair required
- [ ] Grading **Available** → the handset leaves the queue and reaches the till
- [ ] Grading **Used** → also sellable, and the condition is recorded
- [ ] Grading **Damaged** → status DAMAGED, still not sellable
- [ ] Grading **Repair required** → status REPAIR, still not sellable
- [ ] Notes typed during inspection appear on the device
- [ ] Inspecting something that is not awaiting inspection is refused
- [ ] Signed in as **staff**: the queue is visible but there is **no Inspect button** — releasing stock for sale is not the counter's decision

## 4. Classification survives the whole path (FR-8.4)

---

The acceptance test for this module. Do it exactly.

- [ ] Buy a handset as **GLOBAL** with **NEW CUT** ticked
- [ ] Sell it, then return it
- [ ] In the inspection queue it still shows **GLOBAL**
- [ ] Grade it **Used**
- [ ] On the device page it **still reads GLOBAL and NEW CUT**, with condition Used recorded separately

Condition and classification are different facts. Grading a handset's condition must never rewrite what kind of device it is.

## 5. The money (FR-8.5)

- ☐ Refund to a **payment method** — money goes back out
- ☐ Refund to the **customer's account** — no cash moves, they owe less
- ☐ A walk-in cannot have their account credited; it is refused
- ☐ A **deduction** (restocking fee) reduces what is handed back
- ☐ **The one that matters:** a customer who paid in full, returned the goods and got cash back ends with a **zero balance** — not in credit. Check Customers → their Statement
- ☐ Returning against an **unpaid** bill just reduces what they owe

## 6. Trade-in (FR-9.1 – FR-9.3)

On the billing screen, with something in the cart:

- ☐ **Trade-in** → **Add** takes IMEI, type, storage, colour, battery, condition notes, estimated value and agreed value
- ☐ Accepting it adds the old handset to **this branch's stock** immediately
- ☐ Its device page shows the type you chose, including GLOBAL/NEW CUT
- ☐ Its history starts with a **PURCHASED** event
- ☐ Its purchase price is the **agreed value**
- ☐ It is marked as sold here, never the other billing system
- ☐ On the bill: **Bill total** stays the full price, and the trade-in shows as a deduction against what is still to pay
- ☐ Save the bill → the difference is what the customer owed
- ☐ The invoice still shows the full price and full GST — a trade-in is not a discount, and the trade-in appears under the total as settlement, beside the cash, with the old handset's IMEI
- ☐ **Take one in for the full price of the phone and pay nothing.** The bill saves, reads **PAID**, and shows no outstanding balance and no due date. *A bill settled with the handset must not leave a receivable to chase*
- ☐ Do the same for a **named customer**: their statement shows the sale, the trade-in against it, and a balance of zero
- ☐ The sale list and the customer dues screen both read that bill as PAID — settlement is answered in one place, so they cannot disagree
- ☐ Trading in with **no customer** on the bill is allowed when the handset covers the whole total; it is refused only when something is left owing

### 6b. Refunding no more than was taken

- ☐ Sell on **credit** to a customer, pay nothing, then return the goods and choose to refund to **Cash** → the refund is ₹0 and their balance goes to zero. *They never paid, so nothing goes back out*
- ☐ Sell ₹1,000, pay ₹400, return everything, refund to Cash → ₹400 goes back and the remaining ₹600 comes off their account
- ☐ Same unpaid credit sale, but refund to the **customer's account** → the full value is credited. *A credit note is not money leaving the till*
- ☐ Return the rest of a part-returned bill → the two refunds together never exceed what was paid

## 7. Correcting a device

- [ ] A device page shows **Edit** for admin and manager
- [ ] Signed in as **staff** there is no Edit button
- [ ] Change the **main type** and save
- [ ] The device history shows a **RECLASSIFIED** entry with what changed — corrections are recorded, not silent
- [ ] Change **Billed in** from the other system to ECITY → that handset now appears at the till. *This is the fix for NEW stock that was invisible*
- [ ] NEW CUT on anything that is not GLOBAL is refused
- [ ] A **sold** device cannot be edited — an issued invoice describes it
- [ ] The IMEI is shown but not editable

## What M6 deliberately does not include

- **No voiding a return.** Take a new sale if goods go back out
- **No repair workflow** — REPAIR is a status, not a job card
- **No automatic valuation** of trade-ins; the shop decides the number
- **No returns against the other billing system's sales** — those bills do not exist here until M12 imports them

## Housekeeping

The end-to-end tests create real branches and customers in your development database. After a test run you will see extra rows. To get back to a clean state:

```
dropdb ecity && createdb ecity
psql -d ecity -c "CREATE ROLE ecity WITH LOGIN PASSWORD 'ecity' SUPERUSER;"
npm run db:migrate && npm run db:seed
```

## Sign-off

| Tested by      |             |
|----------------|-------------|
| Date           |             |
| Build / commit |             |
| Result         | pass / fail |
| Issues raised  |             |